

AI Assisted Coding

Lab Assignment - 16.1

Name : Likhith Sai

Hall.t.No. : 2303A51026

Batch No. : 01

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

Lab Objectives:

- Understand schema design principles for relational databases.
- Use AI-assisted tools to generate SQL schema definitions.
- Write and refine SQL queries (CRUD + aggregate operations).
- Explore how AI can optimize queries and ensure normalization.

Task 1 – Student Information System Schema

Task: Design a database schema for a Student Information System and generate queries using AI.

Instructions:

- Tables: Students, Courses, Enrollments.
- Define primary keys, foreign keys, and relationships.
- Generate queries:
 - o Insert a new student record.
 - o Fetch all courses enrolled by a student.
 - o Count number of students in each course.

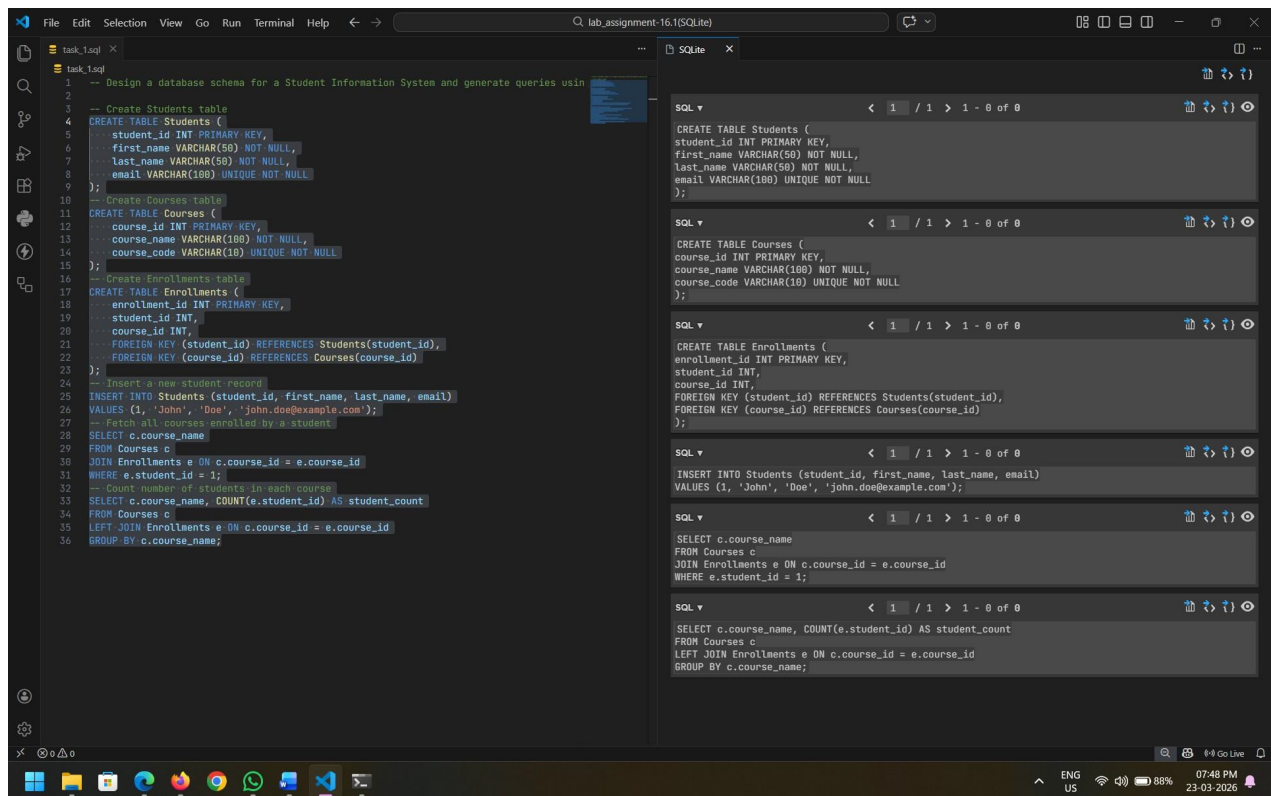
Expected Output:

- SQL CREATE TABLE statements and queries for student– course relationships.

Firstly we have to create a database file :

```
C:\SQLite>sqlite3 task_1.db
SQLite version 3.51.3 2026-03-13 10:38:09
Enter ".help" for usage hints.
sqlite> .database
main: C:\SQLite\task_1.db r/w
sqlite>
```

Then we have to run the .sql file, So that we get the output in this .db file:



The screenshot displays a code editor with an SQL script for a Student Information System. The script includes table creation, data insertion, and various queries. To the right, the SQLite interface shows the execution of these queries, displaying the results of each command.

```
-- Design a database schema for a Student Information System and generate queries using
-- Create Students table
CREATE TABLE Students (
  student_id INT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL
);

-- Create Courses table
CREATE TABLE Courses (
  course_id INT PRIMARY KEY,
  course_name VARCHAR(100) NOT NULL,
  course_code VARCHAR(10) UNIQUE NOT NULL
);

-- Create Enrollments table
CREATE TABLE Enrollments (
  enrollment_id INT PRIMARY KEY,
  student_id INT,
  course_id INT,
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

-- Insert a new student record
INSERT INTO Students (student_id, first_name, last_name, email)
VALUES (1, 'John', 'Doe', 'john.doe@example.com');

-- Fetch all courses enrolled by a student
SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
WHERE e.student_id = 1;

-- Count number of students in each course
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
```

The SQLite interface shows the following results:

- CREATE TABLE Students (student_id INT PRIMARY KEY, first_name VARCHAR(50) NOT NULL, last_name VARCHAR(50) NOT NULL, email VARCHAR(100) UNIQUE NOT NULL);
- CREATE TABLE Courses (course_id INT PRIMARY KEY, course_name VARCHAR(100) NOT NULL, course_code VARCHAR(10) UNIQUE NOT NULL);
- CREATE TABLE Enrollments (enrollment_id INT PRIMARY KEY, student_id INT, course_id INT, FOREIGN KEY (student_id) REFERENCES Students(student_id), FOREIGN KEY (course_id) REFERENCES Courses(course_id));
- INSERT INTO Students (student_id, first_name, last_name, email) VALUES (1, 'John', 'Doe', 'john.doe@example.com');
- SELECT c.course_name FROM Courses c JOIN Enrollments e ON c.course_id = e.course_id WHERE e.student_id = 1;
- SELECT c.course_name, COUNT(e.student_id) AS student_count FROM Courses c LEFT JOIN Enrollments e ON c.course_id = e.course_id GROUP BY c.course_name;

Task 2 – Hospital Management Database

Task: Create schema and queries for a Hospital Management System.

Instructions:

- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).
- Generate queries:
 - List all appointments for a specific doctor.
 - Retrieve patient history by patient ID.
 - Count total patients treated by each doctor.

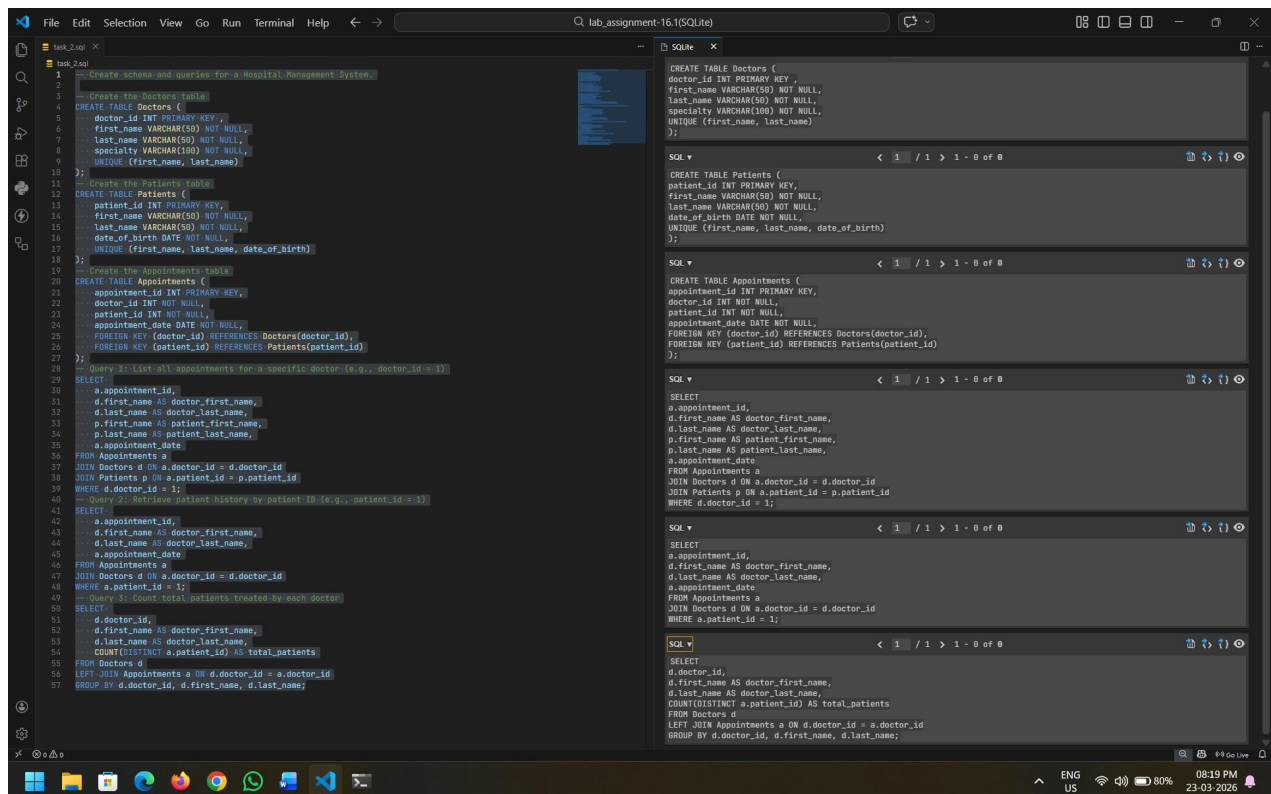
Expected Output:

- Normalized schema and SQL queries with joins.

Firstly we have to create a database file :

```
C:\SQLite>sqlite3 task_2.db
SQLite version 3.51.3 2026-03-13 10:38:09
Enter ".help" for usage hints.
sqlite> .database
main: C:\SQLite\task_2.db r/w
sqlite>
```

Then we have to run the .sql file, So that we get the output in this .db file:



The screenshot displays a SQLite database interface with a file named 'task_2.sql' open. The file contains SQL code for creating a Hospital Management System schema and several queries. The schema includes tables for Doctors, Patients, and Appointments, with their respective fields and constraints. The queries demonstrate various SQL operations such as selecting data, joining tables, and counting records.

```
1 -- Create schema and queries for a Hospital Management System.
2
3 -- Create the Doctors table
4 CREATE TABLE Doctors (
5     doctor_id INT PRIMARY KEY,
6     first_name VARCHAR(50) NOT NULL,
7     last_name VARCHAR(50) NOT NULL,
8     specialty VARCHAR(100) NOT NULL,
9     UNIQUE (first_name, last_name)
10 );
11
12 -- Create the Patients table
13 CREATE TABLE Patients (
14     patient_id INT PRIMARY KEY,
15     first_name VARCHAR(50) NOT NULL,
16     last_name VARCHAR(50) NOT NULL,
17     date_of_birth DATE NOT NULL,
18     UNIQUE (first_name, last_name, date_of_birth)
19 );
20
21 -- Create the Appointments table
22 CREATE TABLE Appointments (
23     appointment_id INT PRIMARY KEY,
24     doctor_id INT NOT NULL,
25     patient_id INT NOT NULL,
26     appointment_date DATE NOT NULL,
27     FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
28     FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
29 );
30
31 -- Query 1: List all appointments for a specific doctor (e.g., doctor_id = 1)
32 SELECT
33     a.appointment_id,
34     d.first_name AS doctor_first_name,
35     d.last_name AS doctor_last_name,
36     p.first_name AS patient_first_name,
37     p.last_name AS patient_last_name,
38     a.appointment_date
39 FROM Appointments a
40 JOIN Doctors d ON a.doctor_id = d.doctor_id
41 JOIN Patients p ON a.patient_id = p.patient_id
42 WHERE d.doctor_id = 1;
43
44 -- Query 2: Retrieve patient history by patient ID (e.g., patient_id = 1)
45 SELECT
46     a.appointment_id,
47     d.first_name AS doctor_first_name,
48     d.last_name AS doctor_last_name,
49     a.appointment_date
50 FROM Appointments a
51 JOIN Doctors d ON a.doctor_id = d.doctor_id
52 WHERE a.patient_id = 1;
53
54 -- Query 3: Count total patients treated by each doctor
55 SELECT
56     d.doctor_id,
57     d.first_name AS doctor_first_name,
58     d.last_name AS doctor_last_name,
59     COUNT(DISTINCT a.patient_id) AS total_patients
60 FROM Doctors d
61 LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
62 GROUP BY d.doctor_id, d.first_name, d.last_name;
```

Task 3 – Library Database

Task: Design schema for a Library Management System.

Instructions:

- Tables: Books, Members, Loans.
- Use AI to suggest indexing strategy for faster queries.
- Generate queries:

- o Retrieve all books currently issued.
- o Find overdue books (loan date > 30 days).
- o Count number of books loaned by each member.

Expected Output:

- Schema + SQL queries demonstrating joins and conditions.

Firstly we have to create a database file :

```
C:\SQLite>sqlite3 task_3.db
SQLite version 3.51.3 2026-03-13 10:38:09
Enter ".help" for usage hints.
sqlite> .database
main: C:\SQLite\task_3.db r/w
sqlite>
```

Then we have to run the .sql file, So that we get the output in this .db file:

```

1  -- Design schema for a Library Management System.
2
3  -- Schema Design for Library Management System
4  CREATE TABLE Books (
5      BookID INT PRIMARY KEY,
6      Title VARCHAR(255) NOT NULL,
7      Author VARCHAR(255) NOT NULL,
8      PublishedDate DATE,
9      ISBN VARCHAR(20) UNIQUE
10 );
11
12 CREATE TABLE Members (
13     MemberID INT PRIMARY KEY,
14     FirstName VARCHAR(255) NOT NULL,
15     LastName VARCHAR(255) NOT NULL,
16     MembershipDate DATE
17 );
18
19 CREATE TABLE Loans (
20     LoanID INT PRIMARY KEY,
21     BookID INT,
22     MemberID INT,
23     LoanDate DATE,
24     ReturnDate DATE,
25     FOREIGN KEY (BookID) REFERENCES Books(BookID),
26     FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
27 );
28
29 -- Indexing Strategy
30 -- To optimize query performance, we can create indexes on the following columns:
31 -- 1. Books(Title) - for faster search by book title.
32 -- 2. Loans(LoanDate) - for efficient retrieval of overdue books.
33
34 CREATE INDEX idx_books_title ON Books(Title);
35 CREATE INDEX idx_loans_loan_date ON Loans(LoanDate);
36
37 -- SQL Queries
38 -- 1. Retrieve all books currently issued (i.e., not returned yet).
39 SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
40 FROM Loans l
41 JOIN Books b ON l.BookID = b.BookID
42 JOIN Members m ON l.MemberID = m.MemberID
43 WHERE l.ReturnDate IS NULL;
44
45 -- 2. Find overdue books (loan date > 30 days)
46 SELECT b.Title, m.FirstName, m.LastName, l.LoanDate
47 FROM Loans l
48 JOIN Books b ON l.BookID = b.BookID
49 JOIN Members m ON l.MemberID = m.MemberID
50 WHERE l.ReturnDate < DATE_SUB(CURDATE(), INTERVAL 30 DAY);
51
52 -- 3. Count number of books loaned by each member.
53 SELECT m.FirstName, m.LastName, COUNT(l.LoanID) AS BooksLoaned
54 FROM Members m
55 LEFT JOIN Loans l ON m.MemberID = l.MemberID
56 GROUP BY m.MemberID;

```

Task 4 – Real-Time Application: Online Shopping Database Scenario: Design a database for an E-commerce platform.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
 - o Retrieve all orders by a user.
 - o Find the most purchased product.
 - o Calculate total revenue in a given month.
- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

Firstly we have to create a database file :

```
C:\SQLite>sqlite3 task_4.db
SQLite version 3.51.3 2026-03-13 10:38:09
Enter ".help" for usage hints.
sqlite> .database
main: C:\SQLite\task_4.db r/w
sqlite>
```

Then we have to run the .sql file, So that we get the output in this .db file:

The screenshot shows a SQL Studio window with a file named 'lab_assignment-16.1(SQLite)'. The left pane displays a .sql file with the following content:

```
1 CREATE TABLE Users (
2   UserID INT PRIMARY KEY,
3   Username VARCHAR(100) NOT NULL,
4   Email VARCHAR(100) NOT NULL,
5   Password VARCHAR(100) NOT NULL,
6   CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
7 );
8
9 CREATE TABLE Products (
10  ProductID INT PRIMARY KEY,
11  ProductName VARCHAR(100) NOT NULL,
12  Price DECIMAL(10, 2) NOT NULL,
13  Stock INT NOT NULL,
14  CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
15 );
16
17 CREATE TABLE Orders (
18  OrderID INT PRIMARY KEY,
19  UserID INT,
20  ProductID INT,
21  TotalAmount DECIMAL(10, 2) NOT NULL,
22  Timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
23 );
24
25 CREATE TABLE OrderDetails (
26  OrderID INT,
27  ProductID INT,
28  Quantity INT NOT NULL,
29  Price DECIMAL(10, 2) NOT NULL,
30  FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
31  FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
32 );
33
34 -- Insert data into Users table
35 INSERT INTO Users (UserID, Username, Email, Password, CreatedAt) VALUES
36 (1, 'JohnDoe', 'john.doe@example.com', 'password123', '2023-03-01 10:00:00'),
37 (2, 'JaneSmith', 'jane.smith@example.com', 'password456', '2023-03-02 11:00:00');
38
39 -- Insert data into Products table
40 INSERT INTO Products (ProductID, ProductName, Price, Stock, CreatedAt) VALUES
41 (1, 'Laptop', 1200, 10, '2023-03-01 09:00:00'),
42 (2, 'Smartphone', 800, 20, '2023-03-01 09:00:00'),
43 (3, 'Tablet', 300, 5, '2023-03-01 09:00:00');
44
45 -- Insert data into Orders table
46 INSERT INTO Orders (OrderID, UserID, ProductID, TotalAmount, Timestamp) VALUES
47 (1, 1, 1, 1200, '2023-03-03 14:30:00'),
48 (2, 2, 2, 800, '2023-03-03 15:00:00'),
49 (3, 1, 3, 300, '2023-03-03 16:00:00');
50
51 -- Insert data into OrderDetails table
52 INSERT INTO OrderDetails (OrderID, ProductID, Quantity, Price, Timestamp) VALUES
53 (1, 1, 1, 1200, '2023-03-03 14:30:00'),
54 (2, 2, 1, 800, '2023-03-03 15:00:00'),
55 (3, 3, 1, 300, '2023-03-03 16:00:00');
```

The right pane displays a .db file with the following content:

```
1 CREATE TABLE Users (
2   UserID INT PRIMARY KEY,
3   Username VARCHAR(100) NOT NULL,
4   Email VARCHAR(100) NOT NULL,
5   Password VARCHAR(100) NOT NULL,
6   CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
7 );
8
9 CREATE TABLE Products (
10  ProductID INT PRIMARY KEY,
11  ProductName VARCHAR(100) NOT NULL,
12  Price DECIMAL(10, 2) NOT NULL,
13  Stock INT NOT NULL,
14  CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
15 );
16
17 CREATE TABLE Orders (
18  OrderID INT PRIMARY KEY,
19  UserID INT,
20  ProductID INT,
21  TotalAmount DECIMAL(10, 2) NOT NULL,
22  Timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
23 );
24
25 CREATE TABLE OrderDetails (
26  OrderID INT,
27  ProductID INT,
28  Quantity INT NOT NULL,
29  Price DECIMAL(10, 2) NOT NULL,
30  FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
31  FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
32 );
33
34 -- Insert data into Users table
35 INSERT INTO Users (UserID, Username, Email, Password, CreatedAt) VALUES
36 (1, 'JohnDoe', 'john.doe@example.com', 'password123', '2023-03-01 10:00:00'),
37 (2, 'JaneSmith', 'jane.smith@example.com', 'password456', '2023-03-02 11:00:00');
38
39 -- Insert data into Products table
40 INSERT INTO Products (ProductID, ProductName, Price, Stock, CreatedAt) VALUES
41 (1, 'Laptop', 1200, 10, '2023-03-01 09:00:00'),
42 (2, 'Smartphone', 800, 20, '2023-03-01 09:00:00'),
43 (3, 'Tablet', 300, 5, '2023-03-01 09:00:00');
44
45 -- Insert data into Orders table
46 INSERT INTO Orders (OrderID, UserID, ProductID, TotalAmount, Timestamp) VALUES
47 (1, 1, 1, 1200, '2023-03-03 14:30:00'),
48 (2, 2, 2, 800, '2023-03-03 15:00:00'),
49 (3, 1, 3, 300, '2023-03-03 16:00:00');
50
51 -- Insert data into OrderDetails table
52 INSERT INTO OrderDetails (OrderID, ProductID, Quantity, Price, Timestamp) VALUES
53 (1, 1, 1, 1200, '2023-03-03 14:30:00'),
54 (2, 2, 1, 800, '2023-03-03 15:00:00'),
55 (3, 3, 1, 300, '2023-03-03 16:00:00');
```

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

- Tables: Students, Subjects, Exams, Results.
- Define primary keys, foreign keys, and referential integrity constraints.
- Generate queries:
 - Insert examination results for a student.
 - Retrieve subject-wise marks for a student.
 - Calculate average marks for each subject.

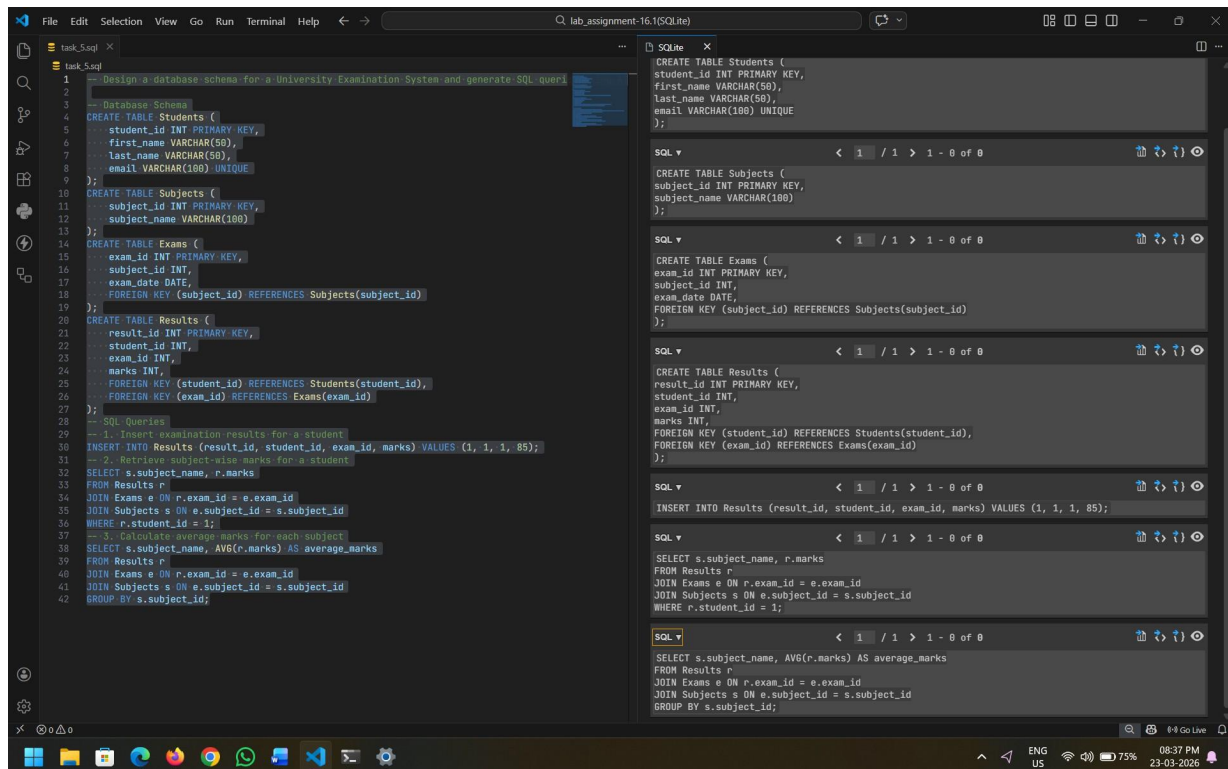
Expected Output:

- Normalized schema and SQL queries involving aggregation and joins.

Firstly we have to create a database file :

```
C:\SQLite>sqlite3 task_5.db
SQLite version 3.51.3 2026-03-13 10:38:09
Enter ".help" for usage hints.
sqlite> .database
main: C:\SQLite\task_5.db r/w
sqlite>
```

Then we have to run the .sql file, So that we get the output in this .db file:



The screenshot shows a SQLite IDE interface with a dark theme. The left pane displays a file named 'task_5.sql' containing a database schema and SQL queries. The right pane shows the execution results of the SQL statements.

task_5.sql

```
1 -- Design a database schema for a University Examination System and generate SQL queries
2
3 -- Database Schema
4 CREATE TABLE Students (
5     student_id INT PRIMARY KEY,
6     first_name VARCHAR(50),
7     last_name VARCHAR(50),
8     email VARCHAR(100) UNIQUE
9 );
10 CREATE TABLE Subjects (
11     subject_id INT PRIMARY KEY,
12     subject_name VARCHAR(100)
13 );
14 CREATE TABLE Exams (
15     exam_id INT PRIMARY KEY,
16     subject_id INT,
17     exam_date DATE,
18     FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)
19 );
20 CREATE TABLE Results (
21     result_id INT PRIMARY KEY,
22     student_id INT,
23     exam_id INT,
24     marks INT,
25     FOREIGN KEY (student_id) REFERENCES Students(student_id),
26     FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
27 );
28
29 -- SQL Queries
30 -- 1. Insert examination results for a student
31 INSERT INTO Results (result_id, student_id, exam_id, marks) VALUES (1, 1, 1, 85);
32 -- 2. Retrieve subject-wise marks for a student
33 SELECT s.subject_name, r.marks
34 FROM Results r
35 JOIN Exams e ON r.exam_id = e.exam_id
36 JOIN Subjects s ON e.subject_id = s.subject_id
37 WHERE r.student_id = 1;
38 -- 3. Calculate average marks for each subject
39 SELECT s.subject_name, AVG(r.marks) AS average_marks
40 FROM Results r
41 JOIN Exams e ON r.exam_id = e.exam_id
42 JOIN Subjects s ON e.subject_id = s.subject_id
43 GROUP BY s.subject_id;
```

SQLite X

```
CREATE TABLE Students (
  student_id INT PRIMARY KEY,
  first_name VARCHAR(50),
  last_name VARCHAR(50),
  email VARCHAR(100) UNIQUE
);

CREATE TABLE Subjects (
  subject_id INT PRIMARY KEY,
  subject_name VARCHAR(100)
);

CREATE TABLE Exams (
  exam_id INT PRIMARY KEY,
  subject_id INT,
  exam_date DATE,
  FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)
);

CREATE TABLE Results (
  result_id INT PRIMARY KEY,
  student_id INT,
  exam_id INT,
  marks INT,
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
);

INSERT INTO Results (result_id, student_id, exam_id, marks) VALUES (1, 1, 1, 85);

SELECT s.subject_name, r.marks
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
WHERE r.student_id = 1;

SELECT s.subject_name, AVG(r.marks) AS average_marks
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
GROUP BY s.subject_id;
```